

1. Analyze
 2. Preprocess data
 3. Feature Engineering
 4. Build Model Pipeline ()
 5. Measure performance of the model.
 6. Which metric is your main metric for this problem and why? What is your main model as well as their params and why?
- Your comment

How can I measure your point:

1. Your function is callable and runs correctly
2. The performance of your model (in full pipeline) is acceptable.
3. Data Analysis is correct or make sense
4. The data preprocessing is correct or make sense
5. The Feature engineering is correct or make sense
6. Model pipeline
7. Any other additional process will be considered a small plus point.

Churn rate is a marketing metric that describes the number of customers who leave a business over a specific time period. Every user is assigned a prediction value that estimates their state of churn at any given time. This value is based on:

User demographic information Browsing behavior Historical purchase data among other information It factors in our unique and proprietary predictions of how long a user will remain a customer. This score is updated every day for all users who have a minimum of one conversion. The values assigned are between 1 and 5.

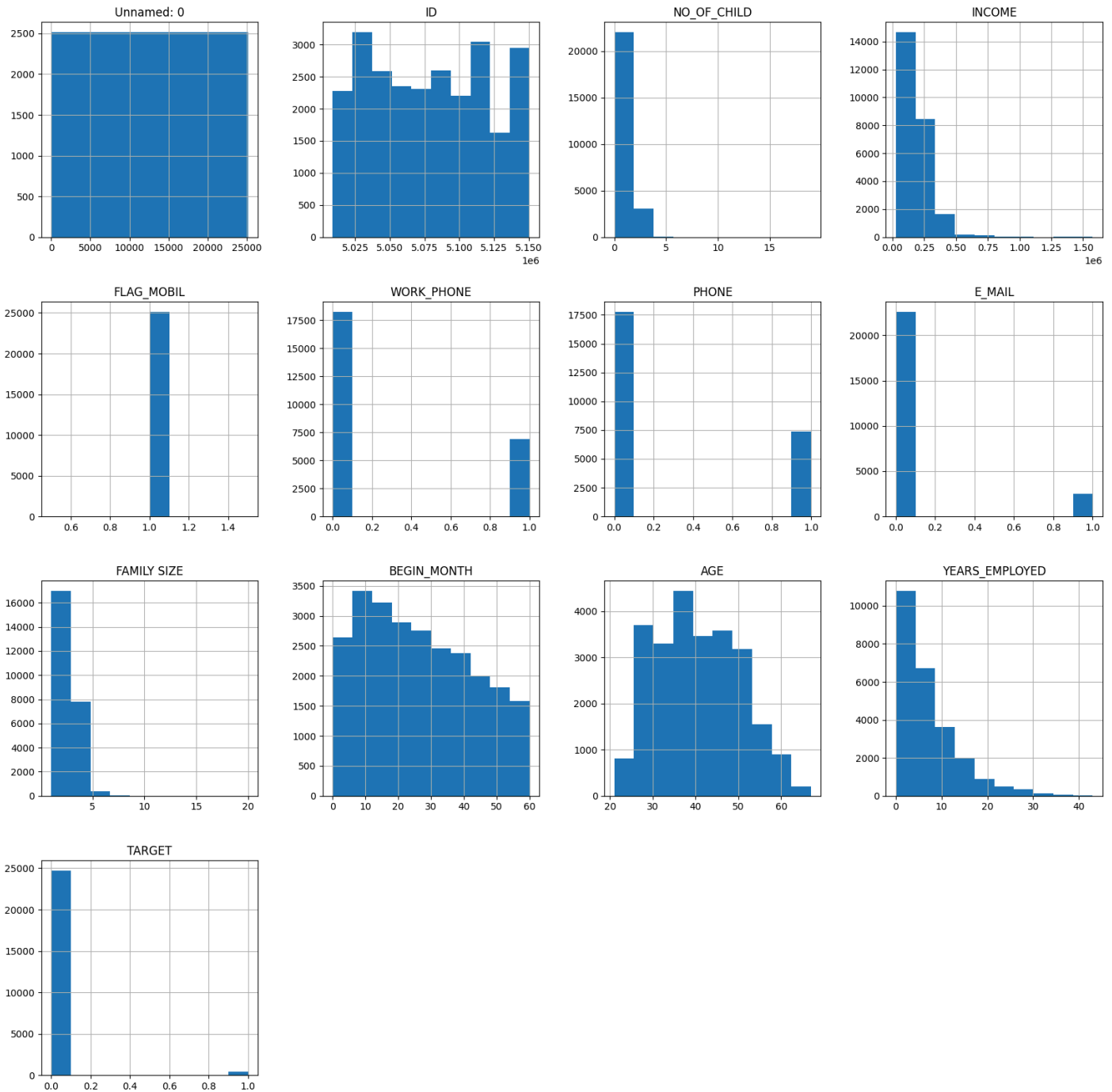
Load Dataset

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import preprocessing
# from google.colab import drive

# drive.mount("/content/drive")
```

```
In [2]: import pandas as pd
PATH = "D:\\Introduction-to-Machine-Learning_MTH10353\\Homework\\6\\credit_fraud.csv" # Path to your file

df = pd.read_csv(PATH)
df.head()
#ToDo: Show histogram of dataframe
df.hist(figsize=(20, 20))
plt.show()
```



Data Analysis

```
In [3]: # Kiểm tra thông tin chung và dữ liệu bị thiếu
display(df.info())
display(df.isnull().sum())

import seaborn as sns
import matplotlib.pyplot as plt

# SỬA Ở ĐÂY: Thay 'churn_score' thành 'TARGET'
plt.figure(figsize=(8, 5))
sns.countplot(x='TARGET', data=df)
plt.title('Distribution of Target (Churn)')
plt.show()

# SỬA Ở ĐÂY: Thêm numeric_only=True để tránh lỗi khi tính toán ma trận tương quan có chứa cột dạng chuỗi
plt.figure(figsize=(12, 10))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Feature Correlation Heatmap')
plt.show()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25134 entries, 0 to 25133
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            25134 non-null  int64
1   ID                    25134 non-null  int64
2   GENDER                25134 non-null  object
3   CAR                   25134 non-null  object
4   REALITY               25134 non-null  object
5   NO_OF_CHILD           25134 non-null  int64
6   INCOME                25134 non-null  float64
7   INCOME_TYPE           25134 non-null  object
8   EDUCATION_TYPE        25134 non-null  object
9   FAMILY_TYPE           25134 non-null  object
10  HOUSE_TYPE            25134 non-null  object
11  FLAG_MOBIL            25134 non-null  int64
12  WORK_PHONE            25134 non-null  int64
13  PHONE                 25134 non-null  int64
14  E_MAIL                25134 non-null  int64
15  FAMILY_SIZE           25134 non-null  float64
16  BEGIN_MONTH           25134 non-null  int64
17  AGE                   25134 non-null  int64
18  YEARS_EMPLOYED        25134 non-null  int64
19  TARGET                25134 non-null  int64

```

```
dtypes: float64(2), int64(11), object(7)
```

```
memory usage: 3.8+ MB
```

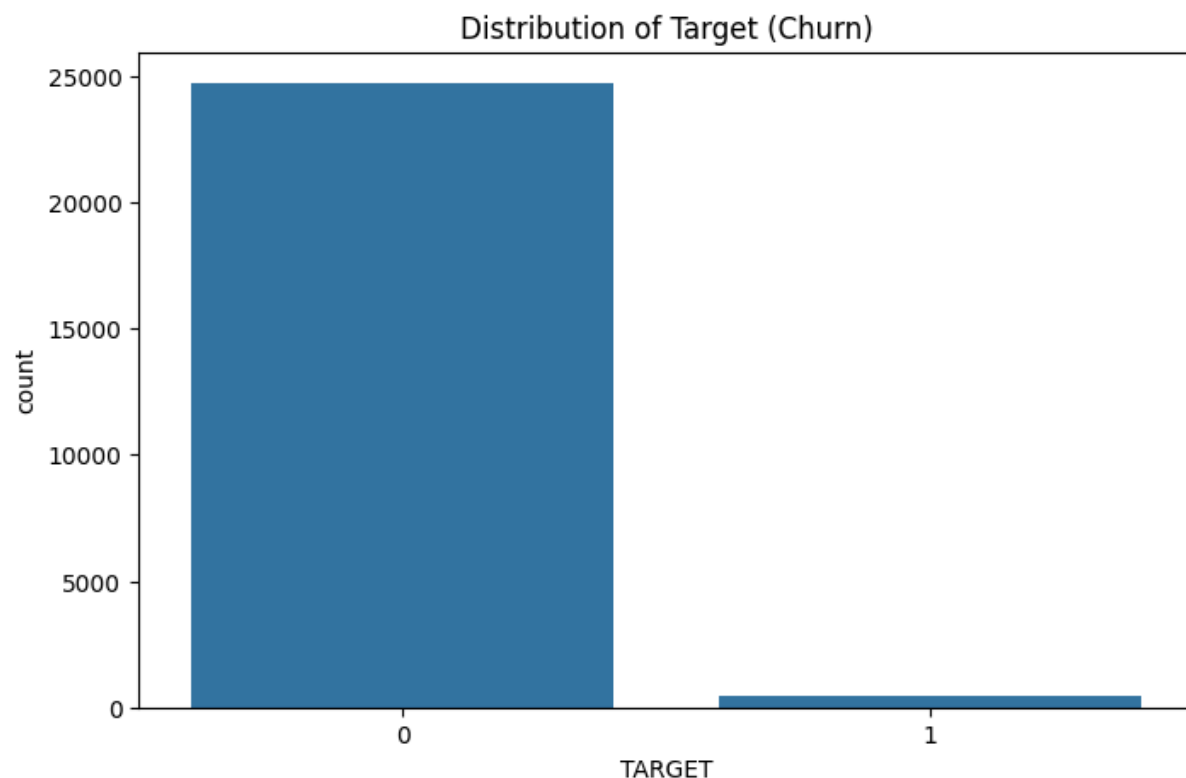
```
None
```

```

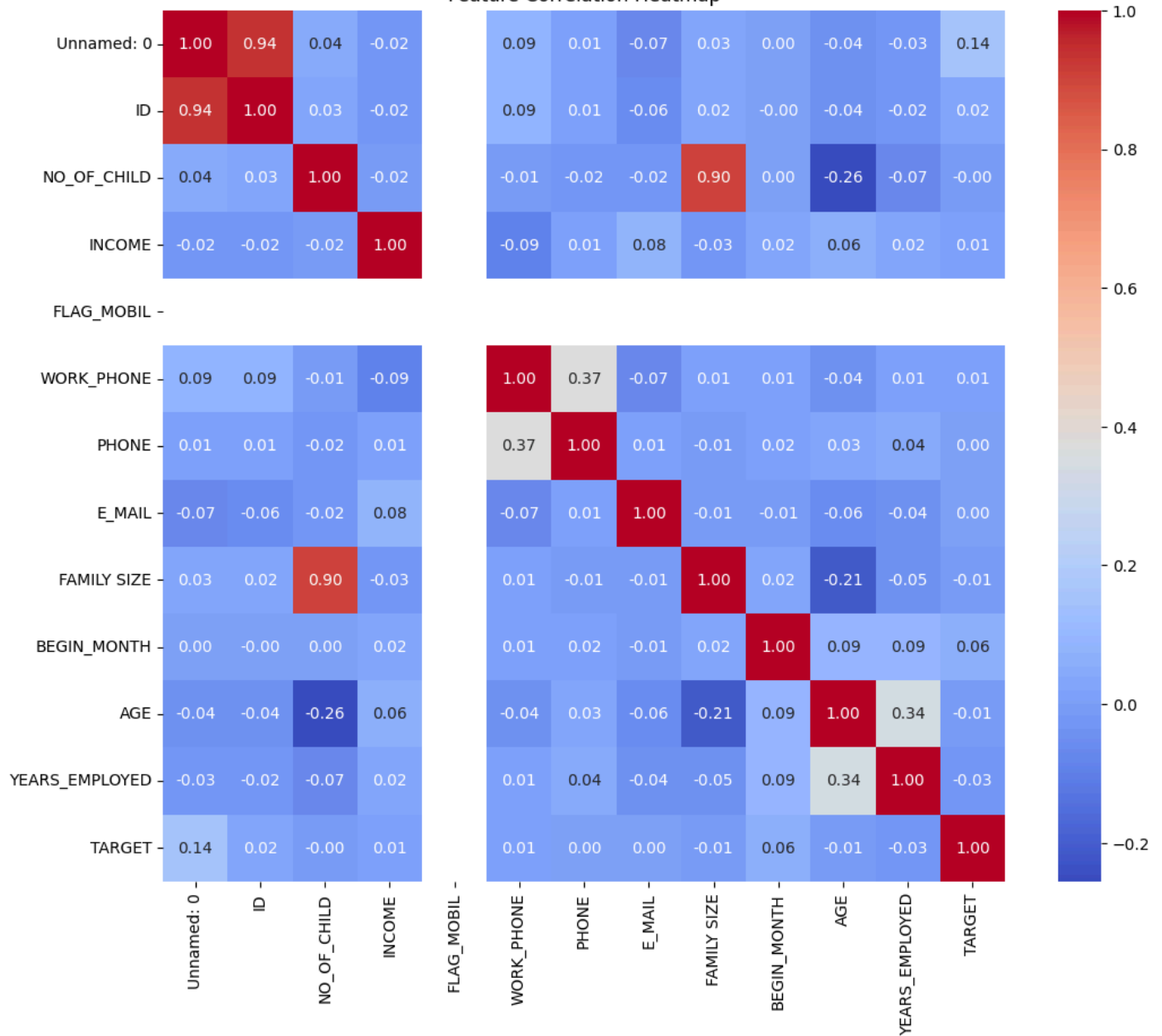
Unnamed: 0      0
ID              0
GENDER          0
CAR             0
REALITY         0
NO_OF_CHILD     0
INCOME          0
INCOME_TYPE     0
EDUCATION_TYPE  0
FAMILY_TYPE     0
HOUSE_TYPE      0
FLAG_MOBIL      0
WORK_PHONE      0
PHONE           0
E_MAIL          0
FAMILY_SIZE     0
BEGIN_MONTH     0
AGE             0
YEARS_EMPLOYED  0
TARGET          0

```

```
dtype: int64
```



Feature Correlation Heatmap



```
In [4]: # Data analysis
        # Todo: analyze your data here
```

```
In [5]: df.head()
```

Out[5]:

	Unnamed: 0	ID	GENDER	CAR	REALITY	NO_OF_CHILD	INCOME	INCOME_TYPE	EDUCATION_TYPE	FAMILY_TYPE
0	0	5008806	M	Y	Y	0	112500.0	Working	Secondary / secondary special	Married
1	1	5008808	F	N	Y	0	270000.0	Commercial associate	Secondary / secondary special	Single / not married
2	2	5008809	F	N	Y	0	270000.0	Commercial associate	Secondary / secondary special	Single / not married
3	3	5008810	F	N	Y	0	270000.0	Commercial associate	Secondary / secondary special	Single / not married
4	4	5008811	F	N	Y	0	270000.0	Commercial associate	Secondary / secondary special	Single / not married

```
In [6]: df.shape
```

```
Out[6]: (25134, 20)
```

```
In [7]: df.columns
```

```
Out[7]: Index(['Unnamed: 0', 'ID', 'GENDER', 'CAR', 'REALITY', 'NO_OF_CHILD', 'INCOME',
              'INCOME_TYPE', 'EDUCATION_TYPE', 'FAMILY_TYPE', 'HOUSE_TYPE',
              'FLAG_MOBIL', 'WORK_PHONE', 'PHONE', 'E_MAIL', 'FAMILY SIZE',
              'BEGIN_MONTH', 'AGE', 'YEARS_EMPLOYED', 'TARGET'],
              dtype='object')
```

```
In [8]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25134 entries, 0 to 25133
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            25134 non-null  int64
1   ID                    25134 non-null  int64
2   GENDER                25134 non-null  object
3   CAR                   25134 non-null  object
4   REALITY               25134 non-null  object
5   NO_OF_CHILD           25134 non-null  int64
6   INCOME                25134 non-null  float64
7   INCOME_TYPE           25134 non-null  object
8   EDUCATION_TYPE        25134 non-null  object
9   FAMILY_TYPE           25134 non-null  object
10  HOUSE_TYPE            25134 non-null  object
11  FLAG_MOBIL            25134 non-null  int64
12  WORK_PHONE            25134 non-null  int64
13  PHONE                 25134 non-null  int64
14  E_MAIL                25134 non-null  int64
15  FAMILY SIZE           25134 non-null  float64
16  BEGIN_MONTH           25134 non-null  int64
17  AGE                   25134 non-null  int64
18  YEARS_EMPLOYED        25134 non-null  int64
19  TARGET                25134 non-null  int64
dtypes: float64(2), int64(11), object(7)
memory usage: 3.8+ MB
```

```
In [9]: df.describe()
```

Out[9]:

	Unnamed: 0	ID	NO_OF_CHILD	INCOME	FLAG_MOBIL	WORK_PHONE	PHONE	E_MAIL
count	25134.000000	2.513400e+04	25134.000000	2.513400e+04	25134.0	25134.000000	25134.000000	25134.000000
mean	12566.500000	5.078838e+06	0.512334	1.948339e+05	1.0	0.273812	0.292791	0.100660
std	7255.705169	4.194102e+04	0.787785	1.045110e+05	0.0	0.445923	0.455052	0.300885
min	0.000000	5.008806e+06	0.000000	2.700000e+04	1.0	0.000000	0.000000	0.000000
25%	6283.250000	5.042228e+06	0.000000	1.350000e+05	1.0	0.000000	0.000000	0.000000
50%	12566.500000	5.079004e+06	0.000000	1.800000e+05	1.0	0.000000	0.000000	0.000000
75%	18849.750000	5.115604e+06	1.000000	2.250000e+05	1.0	1.000000	1.000000	0.000000
max	25133.000000	5.150487e+06	19.000000	1.575000e+06	1.0	1.000000	1.000000	1.000000

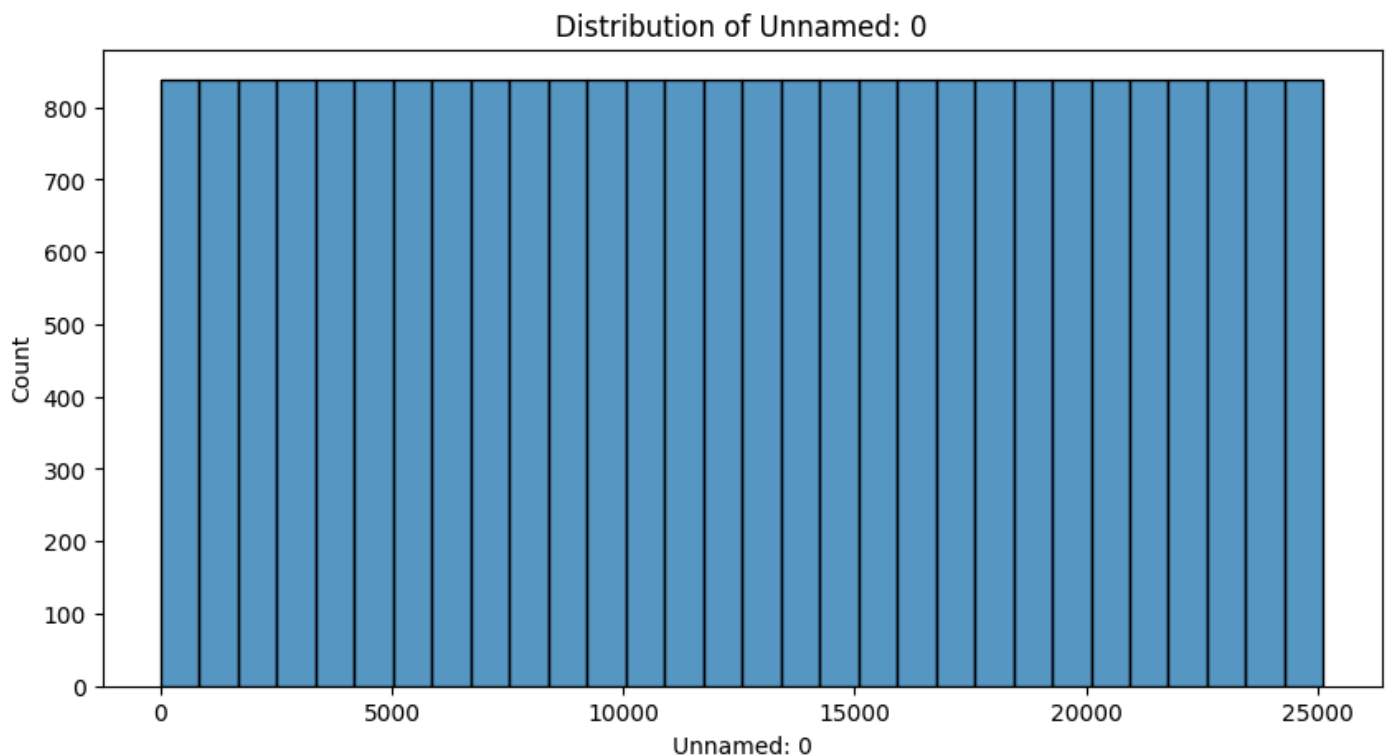
```
In [10]: print("Missing values:")
df.isnull().sum()

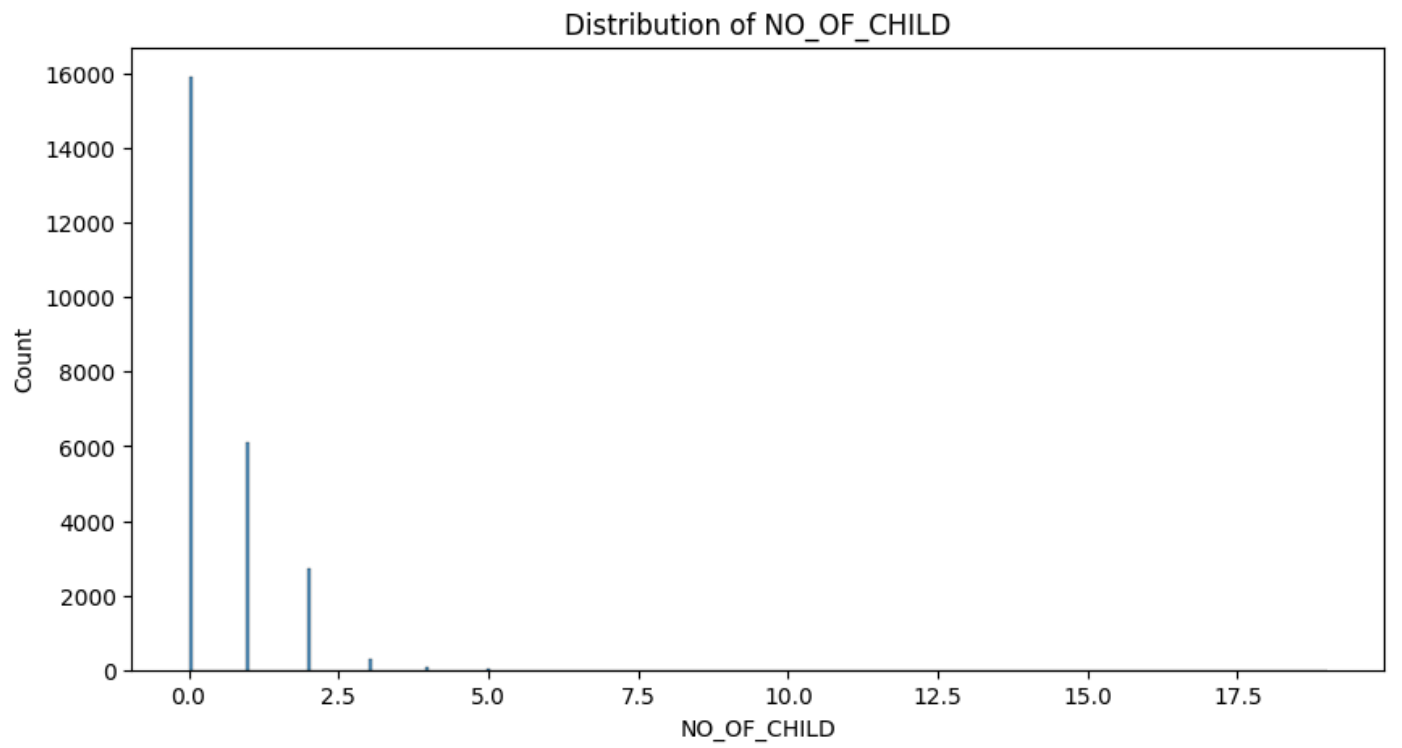
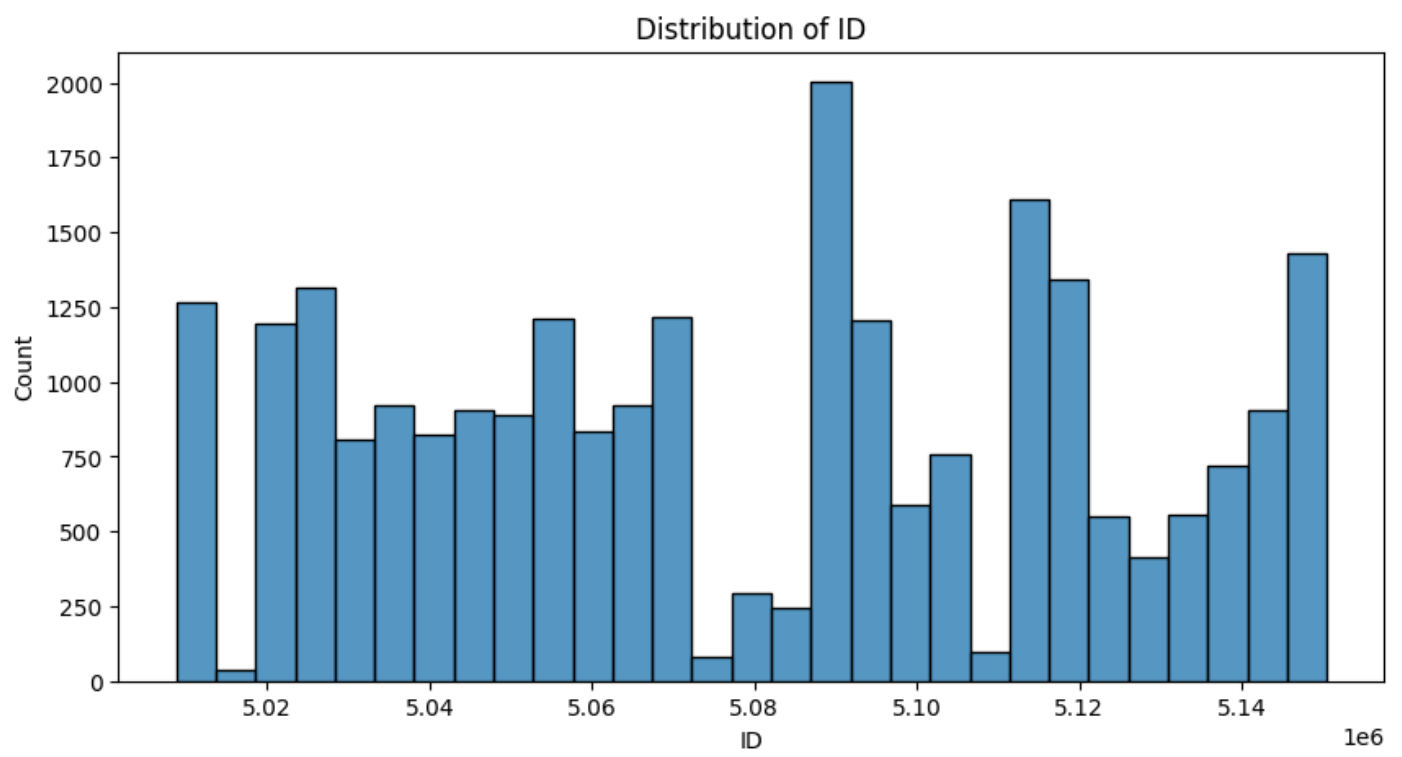
Missing values:
```

```
Out[10]: Unnamed: 0      0
          ID              0
          GENDER          0
          CAR              0
          REALITY         0
          NO_OF_CHILD     0
          INCOME          0
          INCOME_TYPE     0
          EDUCATION_TYPE  0
          FAMILY_TYPE     0
          HOUSE_TYPE      0
          FLAG_MOBIL      0
          WORK_PHONE      0
          PHONE           0
          E_MAIL          0
          FAMILY_SIZE     0
          BEGIN_MONTH     0
          AGE             0
          YEARS_EMPLOYED  0
          TARGET          0
          dtype: int64
```

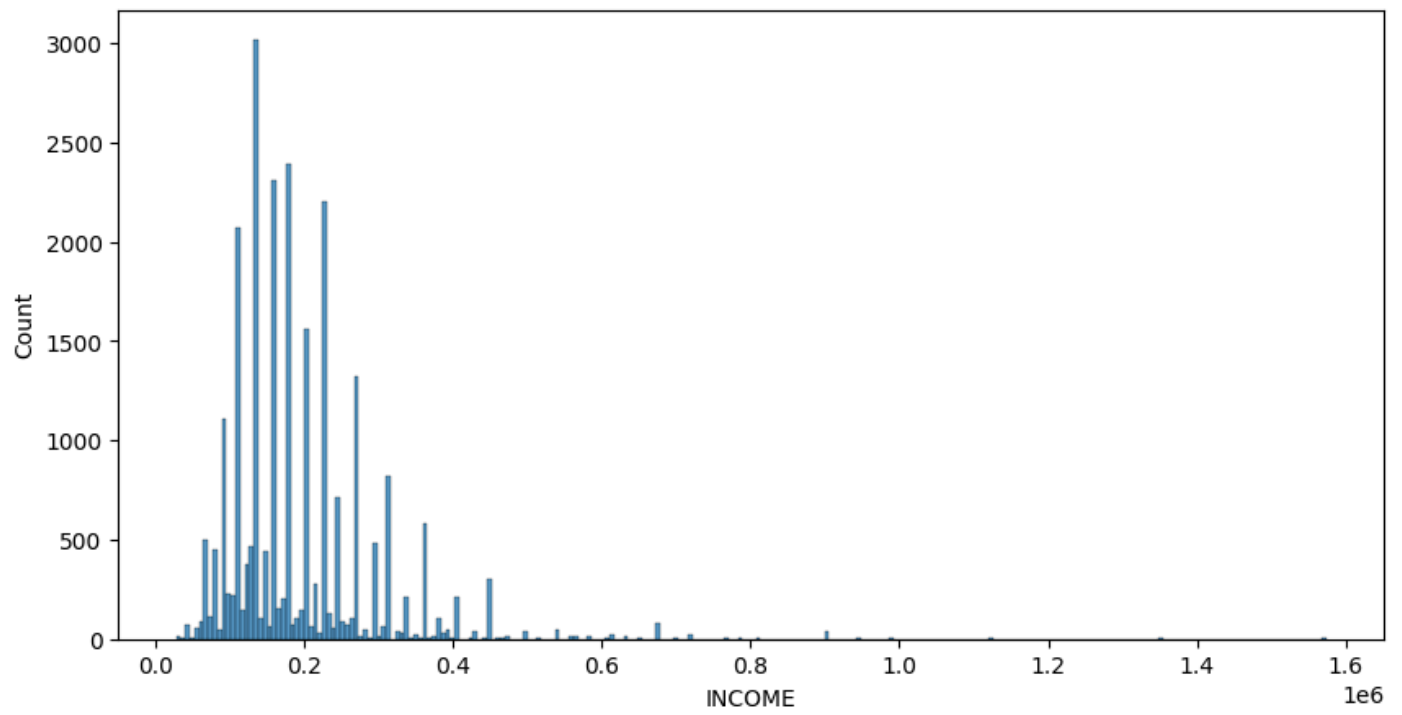
```
In [11]: non_binary_columns = [col for col in df.columns if df[col].nunique() > 2]
         for column in non_binary_columns:
             plt.figure(figsize=(10, 5))
             sns.histplot(df[column])
             plt.title(f"Distribution of {column}")
             plt.show()

         categorical_col=df.select_dtypes(include=['object']).columns
         for column in categorical_col:
             plt.figure(figsize=(6, 4))
             sns.countplot(x=df[column])
             plt.title(f'Count of {column.capitalize()}')
             plt.xlabel(column.capitalize())
             plt.ylabel('Count')
             plt.show()
```

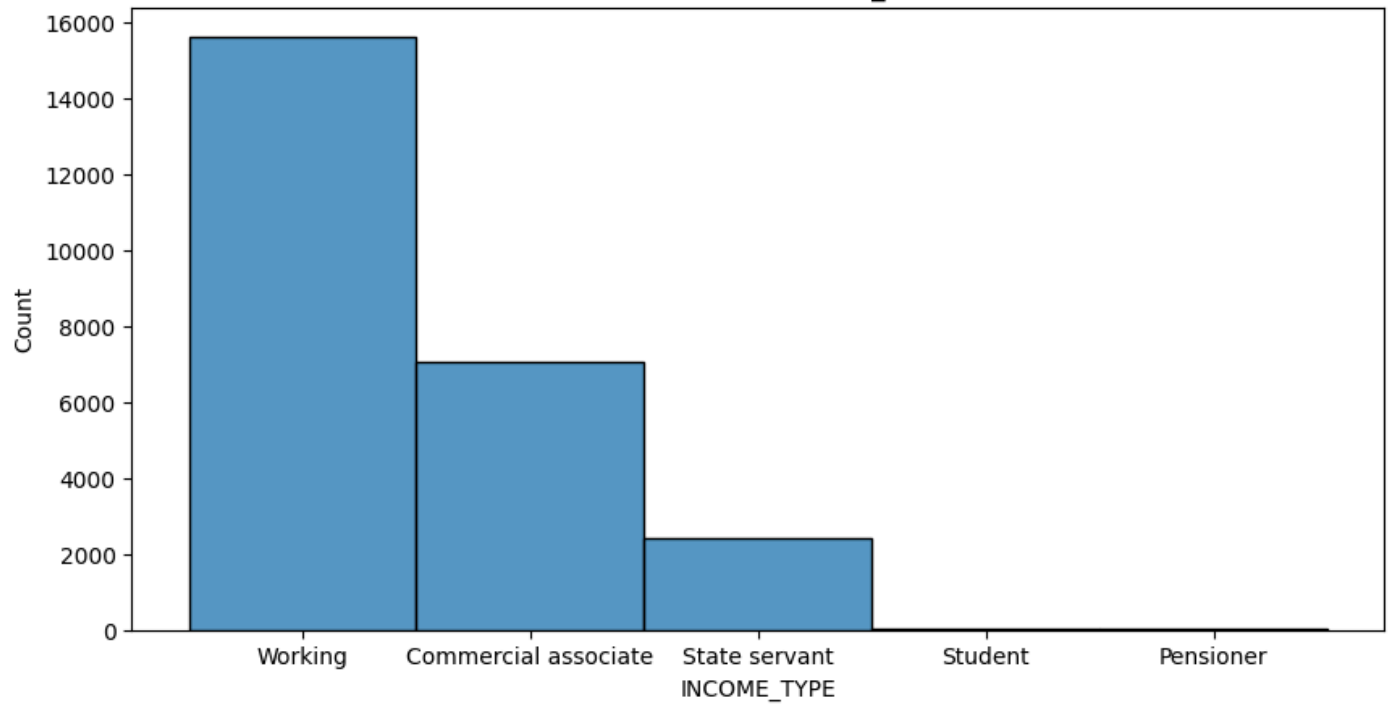




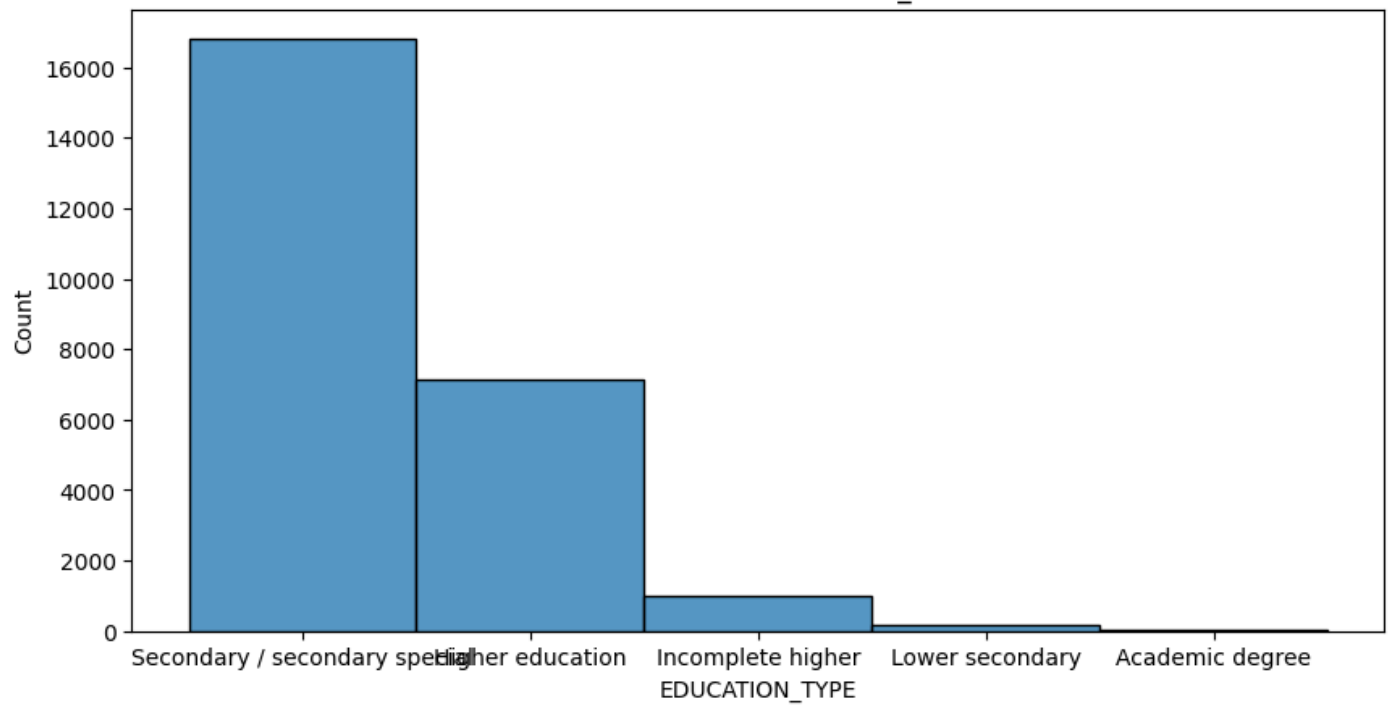
Distribution of INCOME



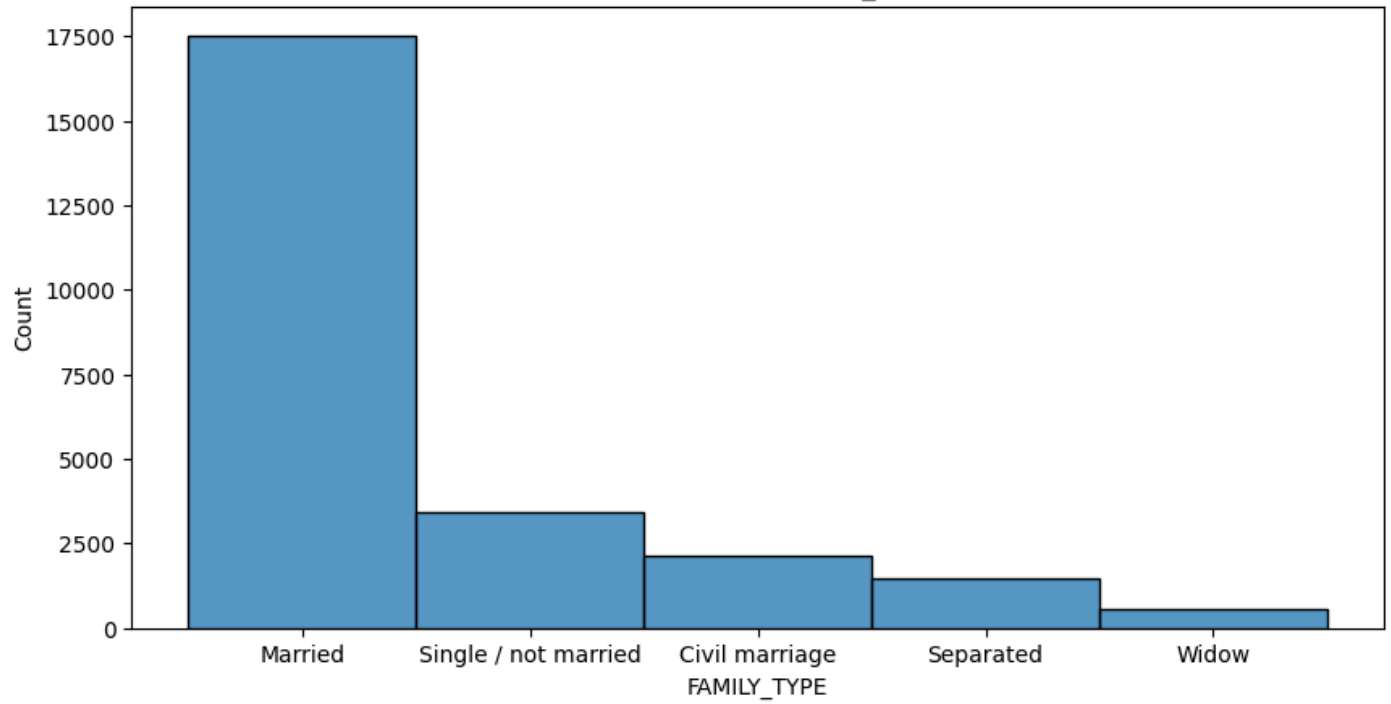
Distribution of INCOME_TYPE



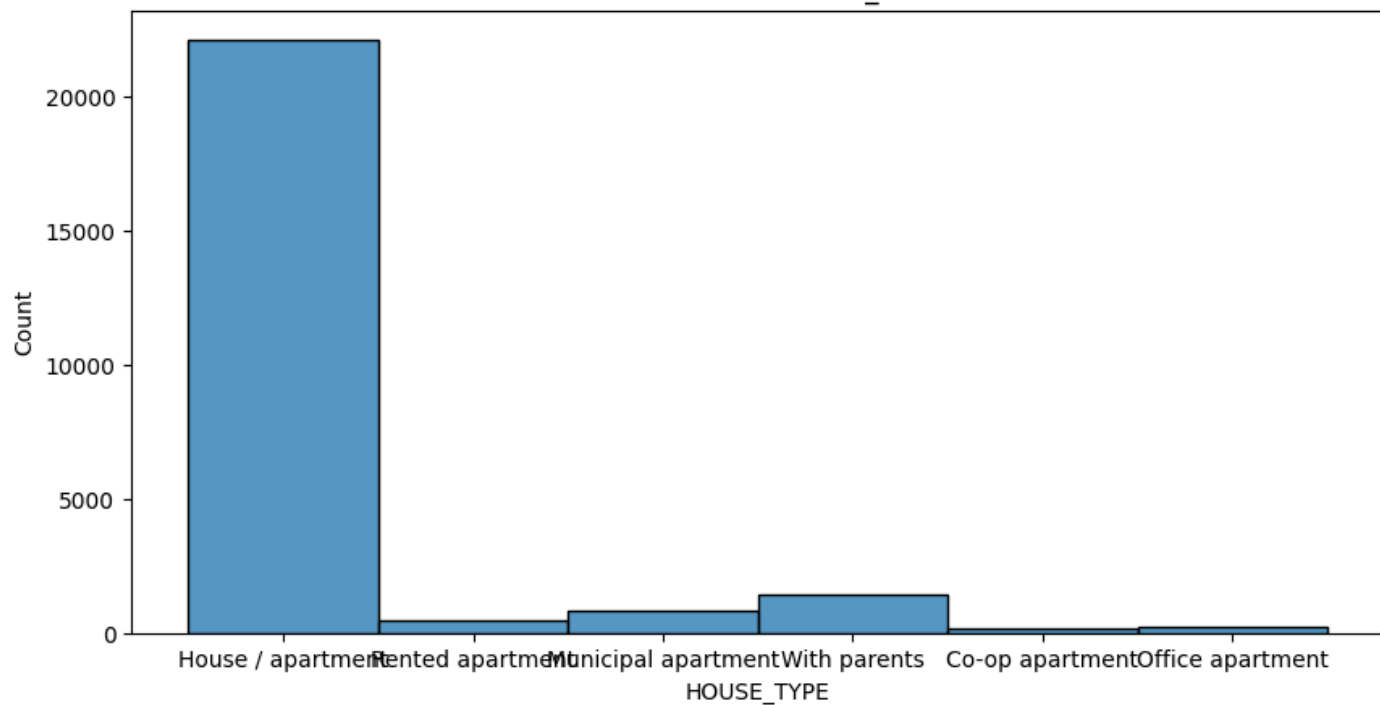
Distribution of EDUCATION_TYPE



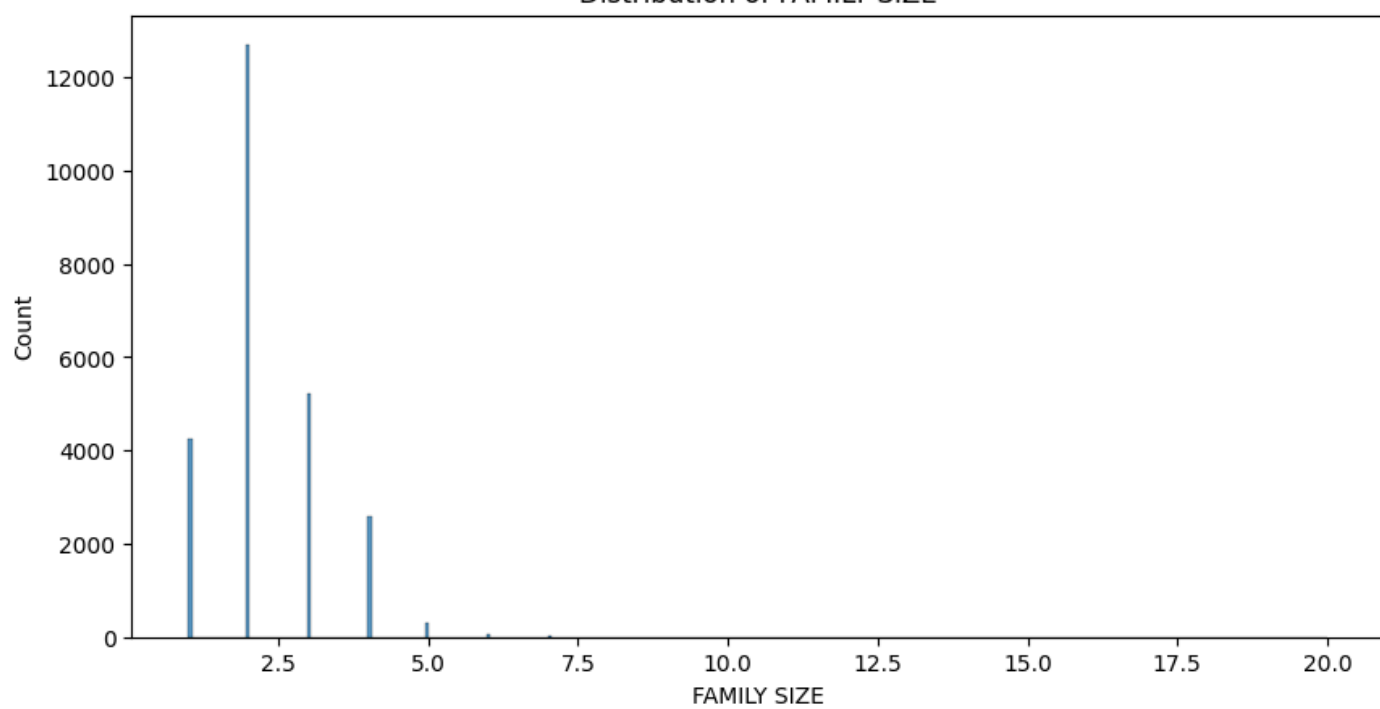
Distribution of FAMILY_TYPE



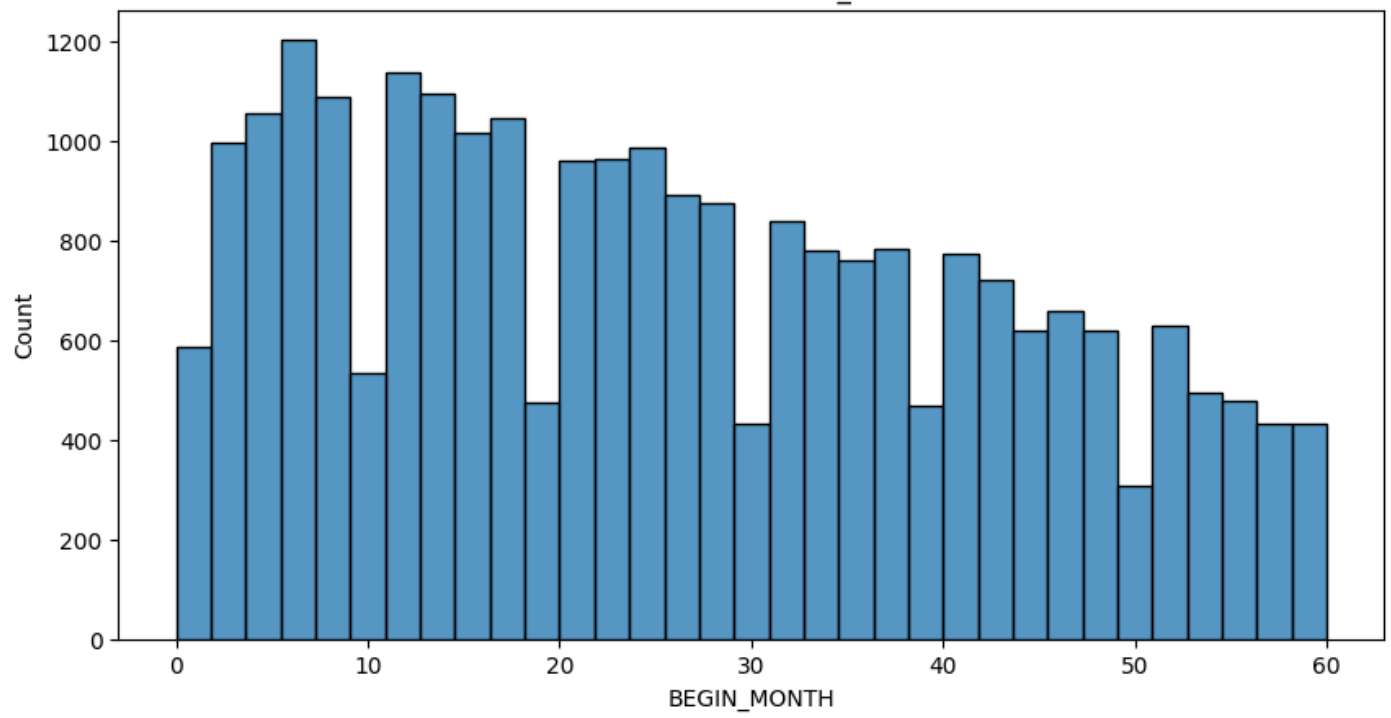
Distribution of HOUSE_TYPE



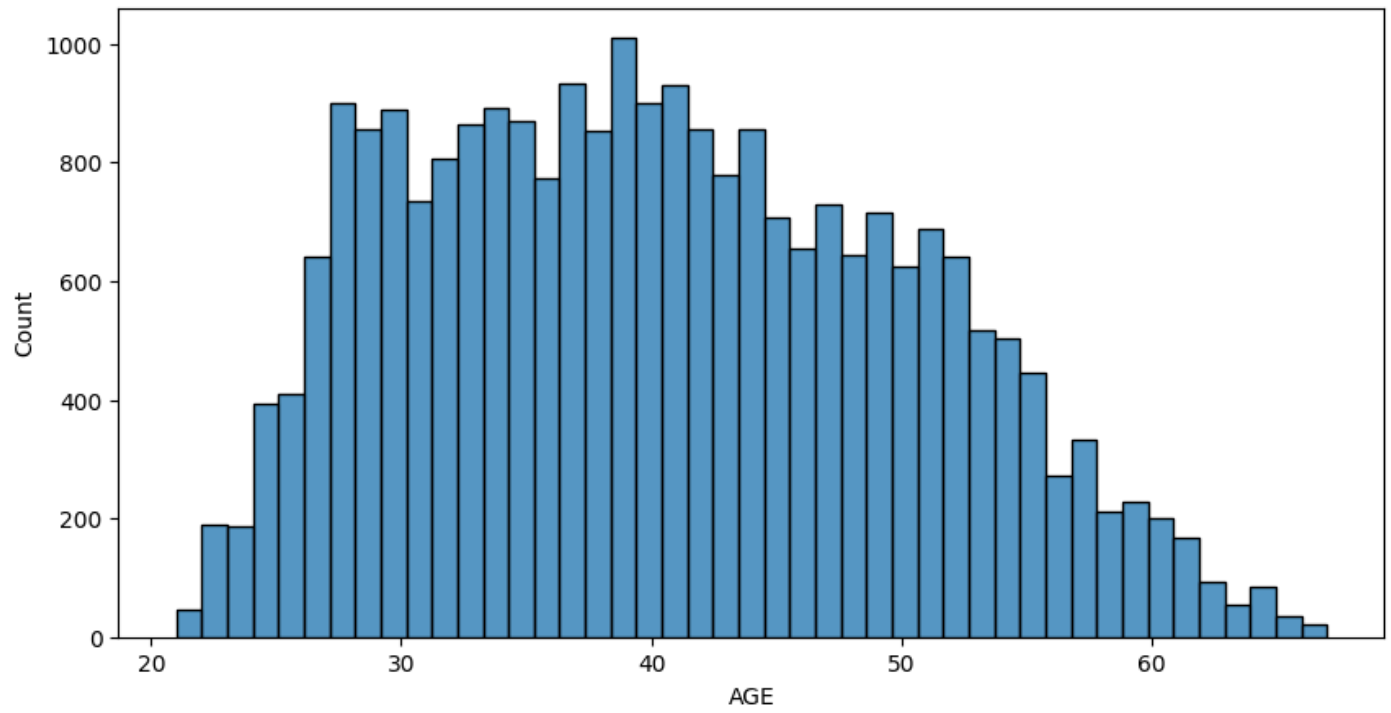
Distribution of FAMILY SIZE



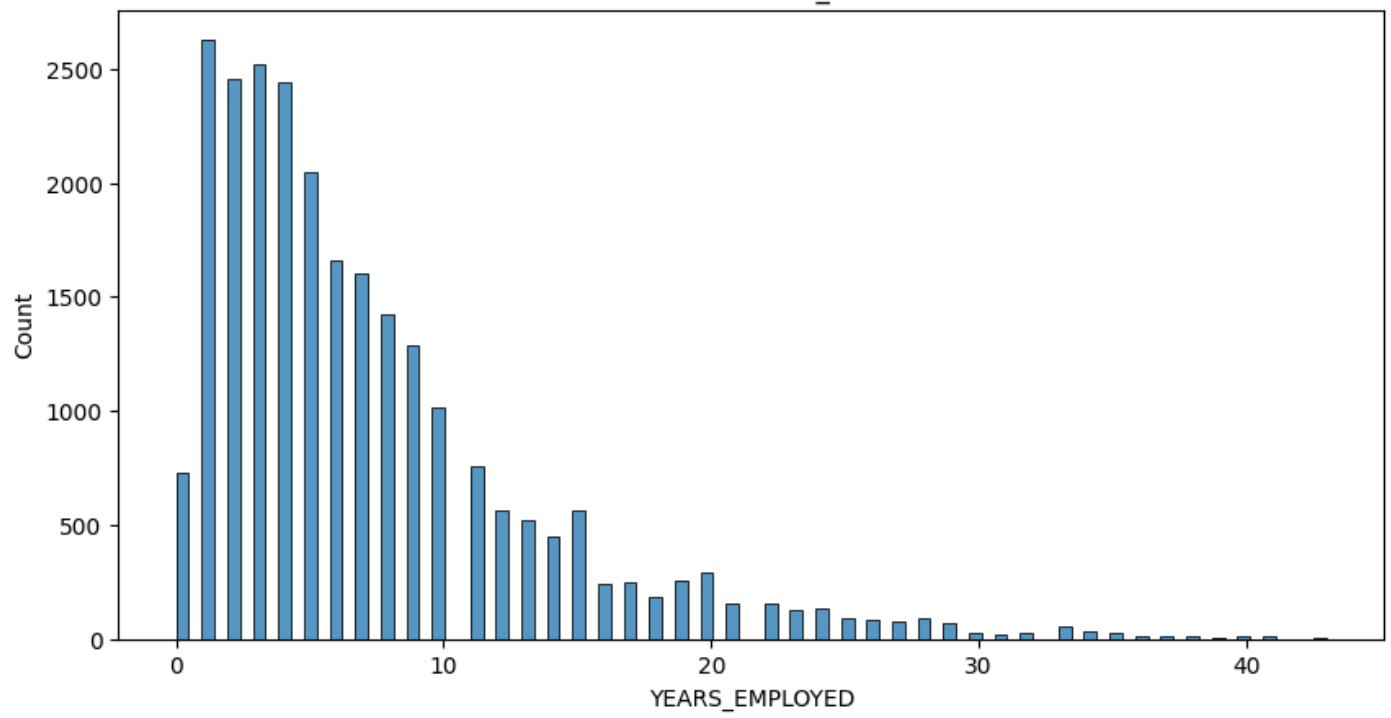
Distribution of BEGIN_MONTH



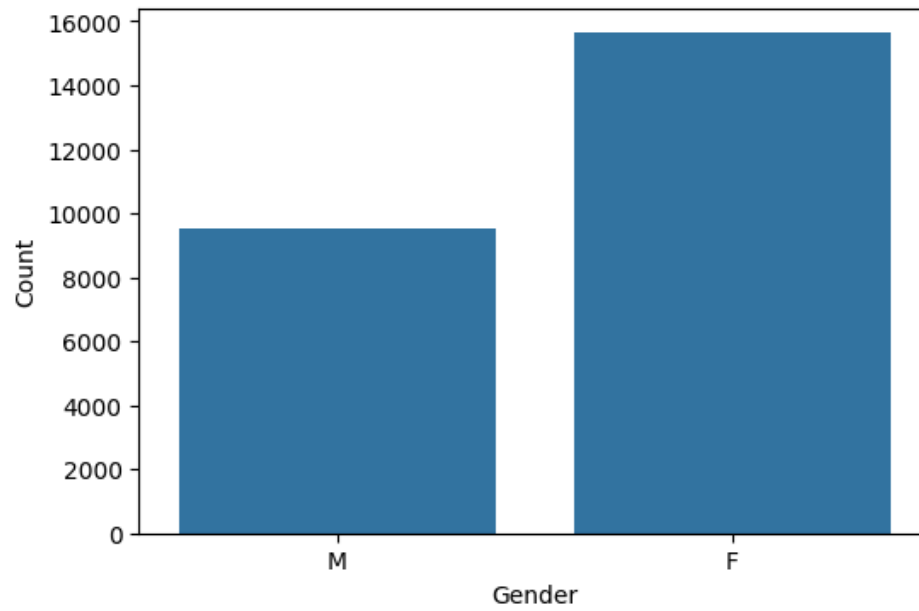
Distribution of AGE



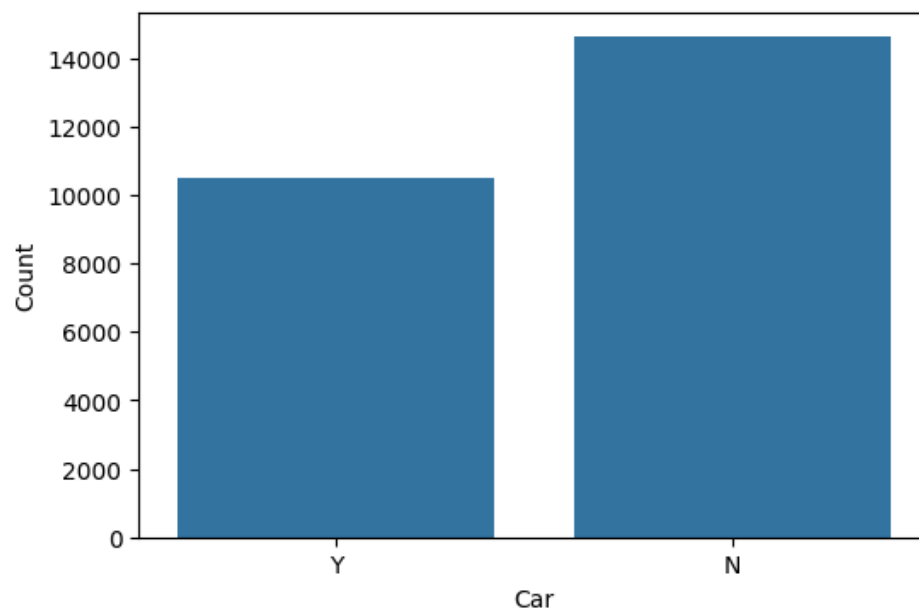
Distribution of YEARS_EMPLOYED

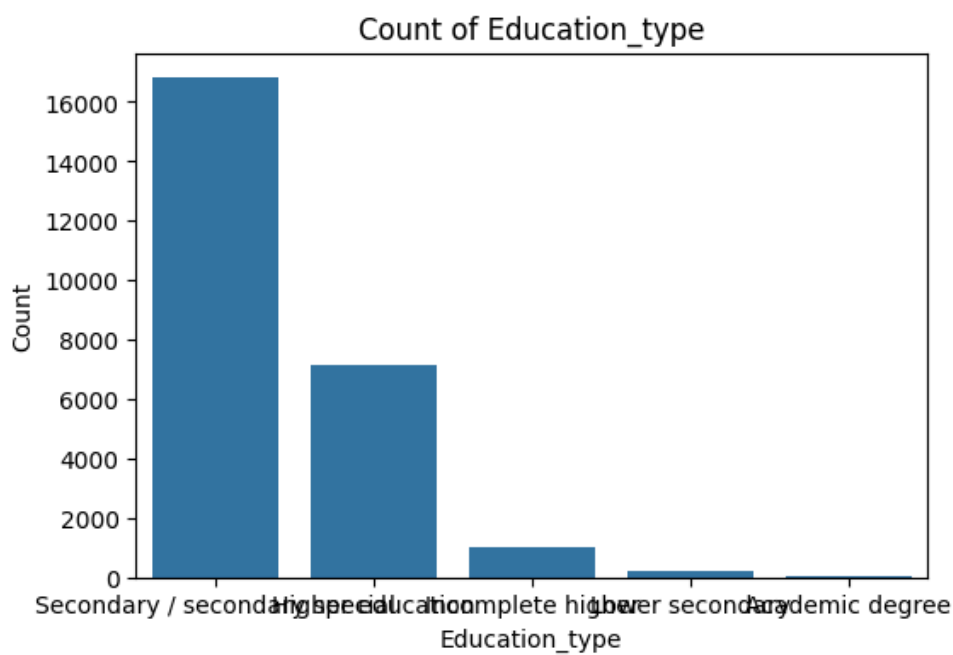
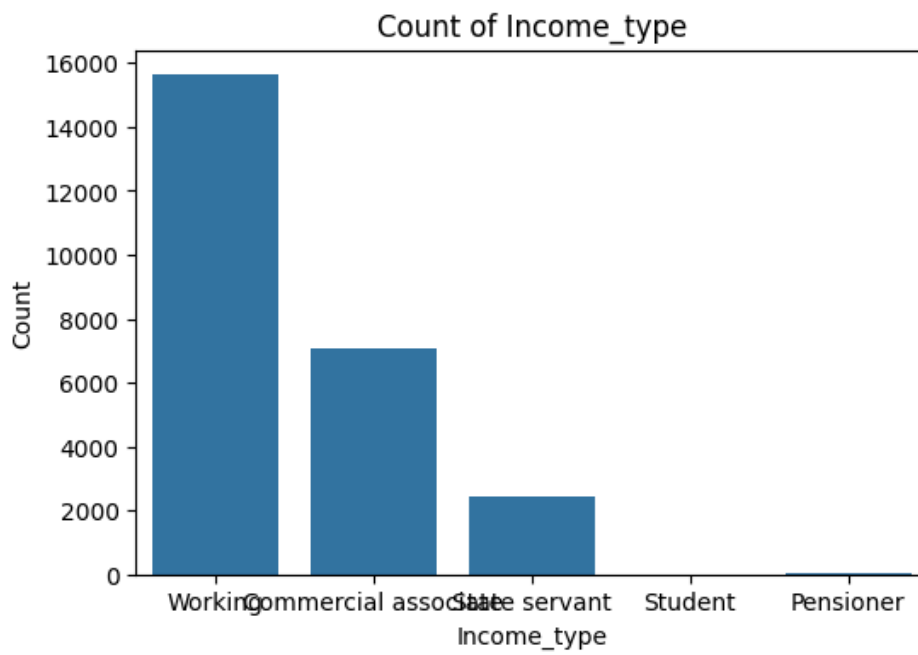
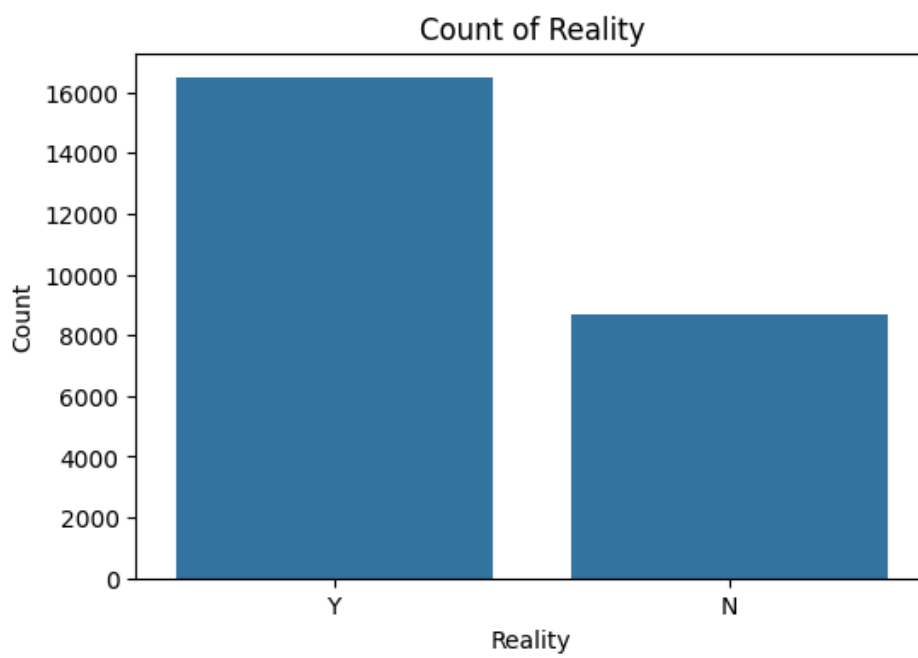


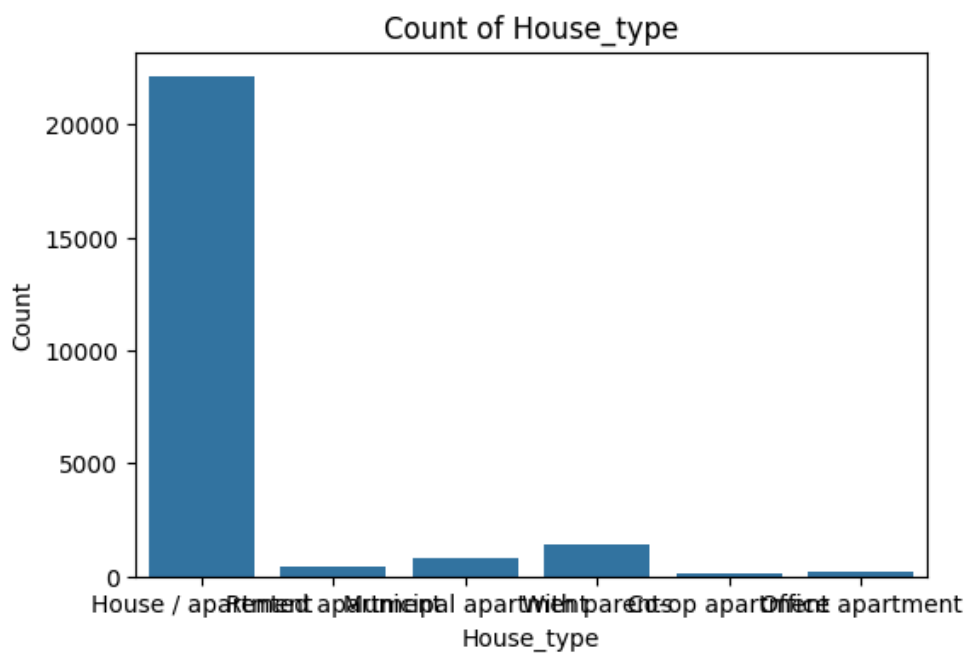
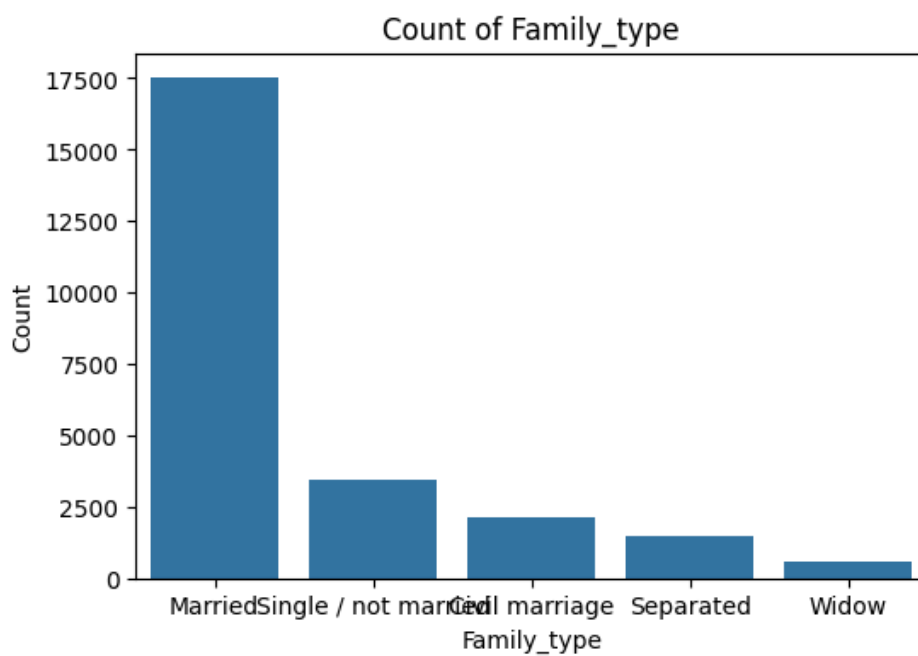
Count of Gender



Count of Car







Preprocessing

```
In [12]: def preprocessing_data(df):
        """
        Preprocess your data (eg. Drop null datapoints or fill missing data)
        :param df: pandas DataFrame
        :return: pandas DataFrame
        """

        # Todo: preprocess data
        df = df.dropna()
        df = df.drop_duplicates()
        df = df[df['NO_OF_CHILD'] < 14]
        return df
```

```
In [13]: df = preprocessing_data(df.copy())
```

Feature Engineering

```
In [14]: # Heatmap
import seaborn as sns

def apply_feature_engineering(df):
    """
    Apply all feature engineering to transform your data into number
    :param df: pandas DataFrame
    :return: pandas DataFrame
```

```

"""
# Todo: Normalize your data
#Feature selection
#Drop unecessary features
df = df.drop(['Unnamed: 0'],axis=1)
df = df.drop(['ID'],axis=1)
df = df.drop(['FLAG_MOBIL'],axis=1)

#Feature transformation
#Label encoding
label_encoder = preprocessing.LabelEncoder()

df['GENDER'] = label_encoder.fit_transform(df['GENDER'])
df['CAR'] = label_encoder.fit_transform(df['CAR'])
df['REALITY'] = label_encoder.fit_transform(df['REALITY'])

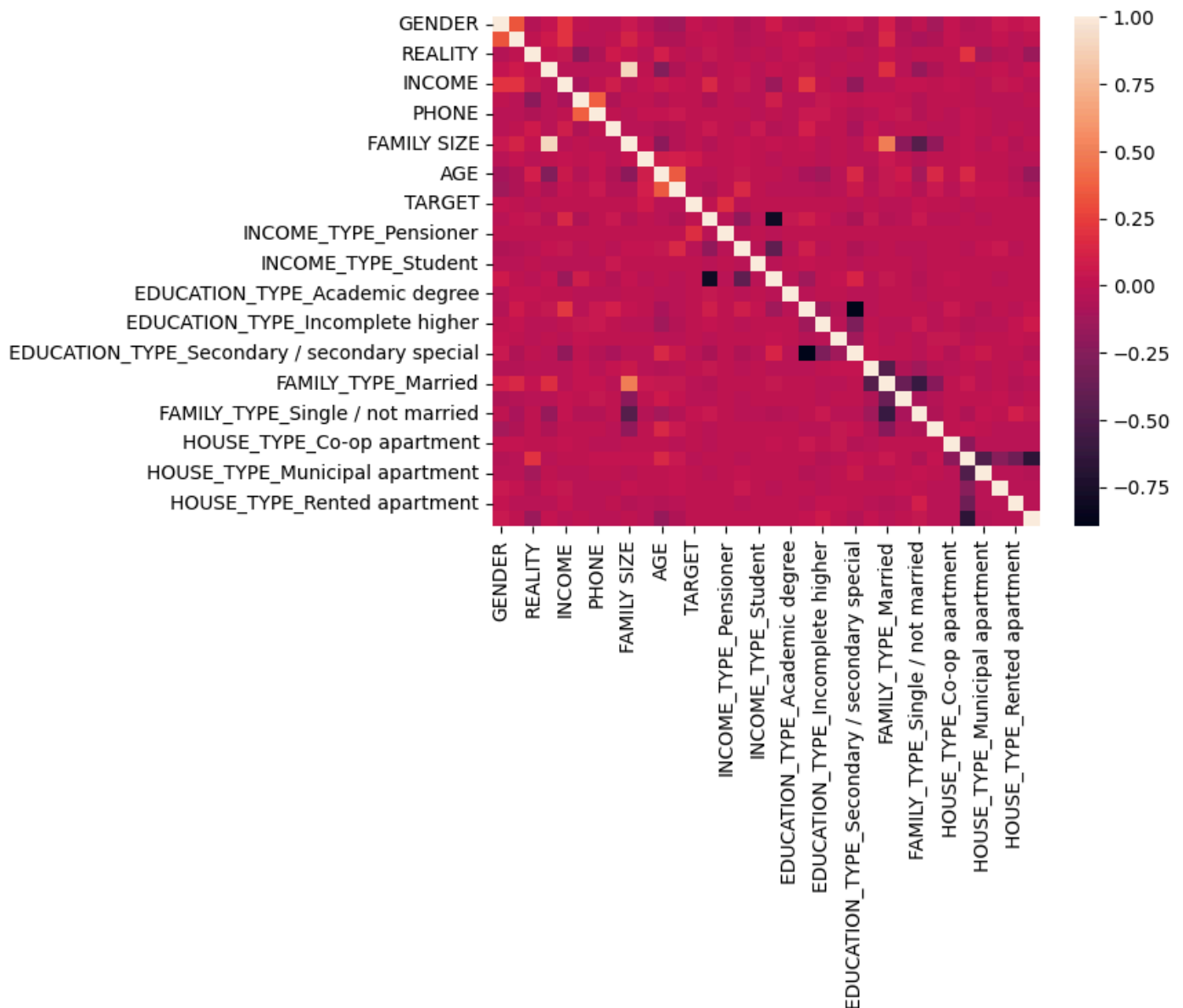
#One-hot encoding
df = pd.get_dummies(df, columns=['INCOME_TYPE'],dtype=int)
df = pd.get_dummies(df, columns=['EDUCATION_TYPE'],dtype=int)
df = pd.get_dummies(df, columns=['FAMILY_TYPE'],dtype=int)
df = pd.get_dummies(df, columns=['HOUSE_TYPE'],dtype=int)

return df

df = apply_feature_engineering(df)
sns.heatmap(df.corr()) # Show heatmap after normalized data

```

Out[14]: <Axes: >



```

In [15]: def prepare_X_y(df):
"""
Feature engineering and create X and y
:param df: pandas dataframe
:return: (X, y) output feature matrix (dataframe), target (series)
"""

```

```
# TODO: Split data into X and y (using sklearn train_test_split). Return two dataframes
columns = df.columns.tolist()
columns.remove('TARGET')

X = df[columns]
y = df['TARGET']
return X, y
```

```
X, y = prepare_X_y(df)
```

Apply machine learning model

Train-test split

```
In [16]: from sklearn.model_selection import train_test_split
RANDOM_STATE = 69
TRAIN_SIZE = 0.7

trainX, testX, trainY, testY = train_test_split(X, y, train_size=TRAIN_SIZE, random_state=RANDOM_STATE)
```

Build SK-learn model

```
In [ ]: from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

def build_model_SVM(X, y):
    """
    Design your model and train it (including your best params)
    """
    pipeline = make_pipeline(
        StandardScaler(),
        SVC(class_weight='balanced', random_state=RANDOM_STATE)
    )

    param_grid = {
        "svc__C": [1, 10, 50, 100],
        "svc__gamma": ["scale", "auto", 0.0001, 0.001, 0.01],
        "svc__kernel": ["linear", "rbf", "poly", "sigmoid"]
    }

    model = GridSearchCV(
        pipeline,
        param_grid,
        cv=5,
        scoring="f1_macro",
        verbose=1,
        n_jobs=-1
    )

    model.fit(X, y)

    print(f"Best params found: {model.best_params_}")

    return model

def calculate_performance(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred, average='weighted')

    print("Accuracy:", acc)
    print("Weighted F1-Score:", f1)
    print("\nClassification Report:\n", classification_report(y_true, y_pred))

    main_score = f1
    return main_score
```



```

model = build_model_SVM(trainX, trainY)
pred = model.predict(testX)
calculate_performance(testY, pred)

```

Fitting 5 folds for each of 80 candidates, totalling 400 fits
 Best params found: {'svc__C': 100, 'svc__gamma': 'scale', 'svc__kernel': 'rbf'}
 Accuracy: 0.9369942963257727
 Weighted F1-Score: 0.9559429435241834

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.94	0.97	7434
1	0.09	0.41	0.15	105
accuracy			0.94	7539
macro avg	0.54	0.68	0.56	7539
weighted avg	0.98	0.94	0.96	7539

Out[]: 0.9559429435241834

```

In [ ]: def get_conclusion():
        # TODO: Please return your answer, conclusion and opinion right here
        return (
            "6. Main metric: F1-Macro / Weighted F1-Score. Because the dataset is highly imbalanced (Class 0 is approxi-  

            Main model & params: SVC with kernel='rbf', C=100, gamma='scale', and class_weight='balanced'. The RBF k-  

            Other conclusion: Despite hyperparameter tuning, the SVM model struggles with the extreme class imbalance.  

            While it achieved a decent recall (0.41) for Class 1, the precision remains very low (0.09), resulting in  

        )

```

In [3]: get_conclusion()

Out[3]: "6. Main metric: F1-Macro / Weighted F1-Score. Because the dataset is highly imbalanced (Class 0 is approximatel
 y 70 times larger than Class 1), Accuracy is a misleading metric (predicting all 0s still yields ~98% accuracy).
 F1-Score balances Precision and Recall for the minority class. \nMain model & params: SVC with kernel='rbf', C=1
 00, gamma='scale', and class_weight='balanced'. The RBF kernel handles non-linear relationships well, and the 'b
 alanced' weight is crucial to penalize misclassifications on the minority class more heavily.\n\nOther conclusio
 n: Despite hyperparameter tuning, the SVM model struggles with the extreme class imbalance. While we achieved a
 decent recall (0.41) for Class 1, the precision remains very low (0.09), resulting in many False Positives. This
 indicates the limitation of standard SVM algorithms on extremely skewed data. To further improve the pipeline, w
 e should apply data-level techniques such as SMOTE (Synthetic Minority Over-sampling Technique) during the prepr
 ocessing step."